

Free Form Testing Agent

Document: 02_architecture.md

Version: 1.0

Status: Draft

Author: Kalpesh

Date: June 2026

1. Purpose

This document defines the complete system architecture of the Free Form Testing Agent (FFTA).

The purpose of the architecture is to build an extensible, modular, offline-first autonomous testing platform capable of:

- Understanding unknown applications.
- Exploring application workflows.
- Making testing decisions.
- Detecting software defects.
- Generating reproducible test cases.
- Learning from previous executions.

The architecture follows the principles of:

- Agentic systems
 - Event-driven design
 - Plugin architecture
 - State machine modeling
 - Graph-based exploration
 - Local AI reasoning
-

2. Architectural Principles

The Free Form Testing Agent follows these principles.

2.1 Offline First

The system must operate completely offline.

Examples:

- Ollama LLMs
 - Local databases
 - Local reports
 - Local state storage
-

2.2 Agent Driven

The system behaves as an autonomous agent rather than a scripted automation framework.

The execution cycle is:

```
graph TD; Observe --> Think; Think --> Plan; Plan --> Execute; Execute --> Validate; Validate --> Learn;
```

Observe
↓
Think
↓
Plan
↓
Execute
↓
Validate
↓
Learn

2.3 Plugin Based

Every major subsystem must be replaceable.

Examples:

- UI drivers
 - Explorers
 - Validators
 - LLM providers
 - Bug detectors
 - Reporters
-

2.4 Event Driven

Components communicate through events rather than direct coupling.

Example:

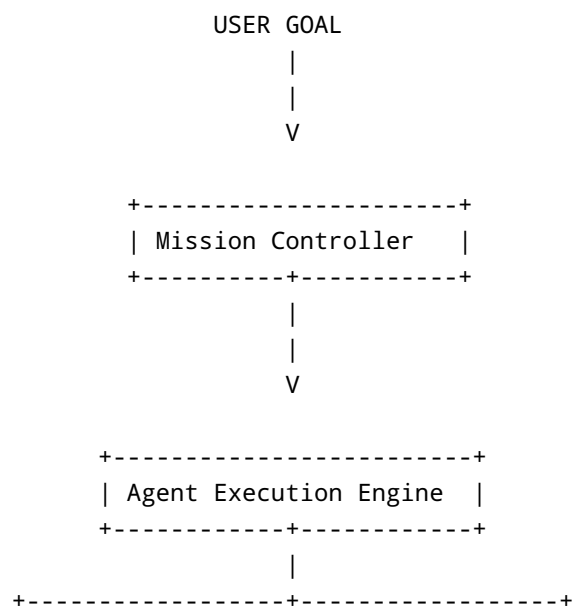
```
ActionExecuted
  ↓
StateObserved
  ↓
ValidationStarted
  ↓
BugDetected
  ↓
TestGenerated
```

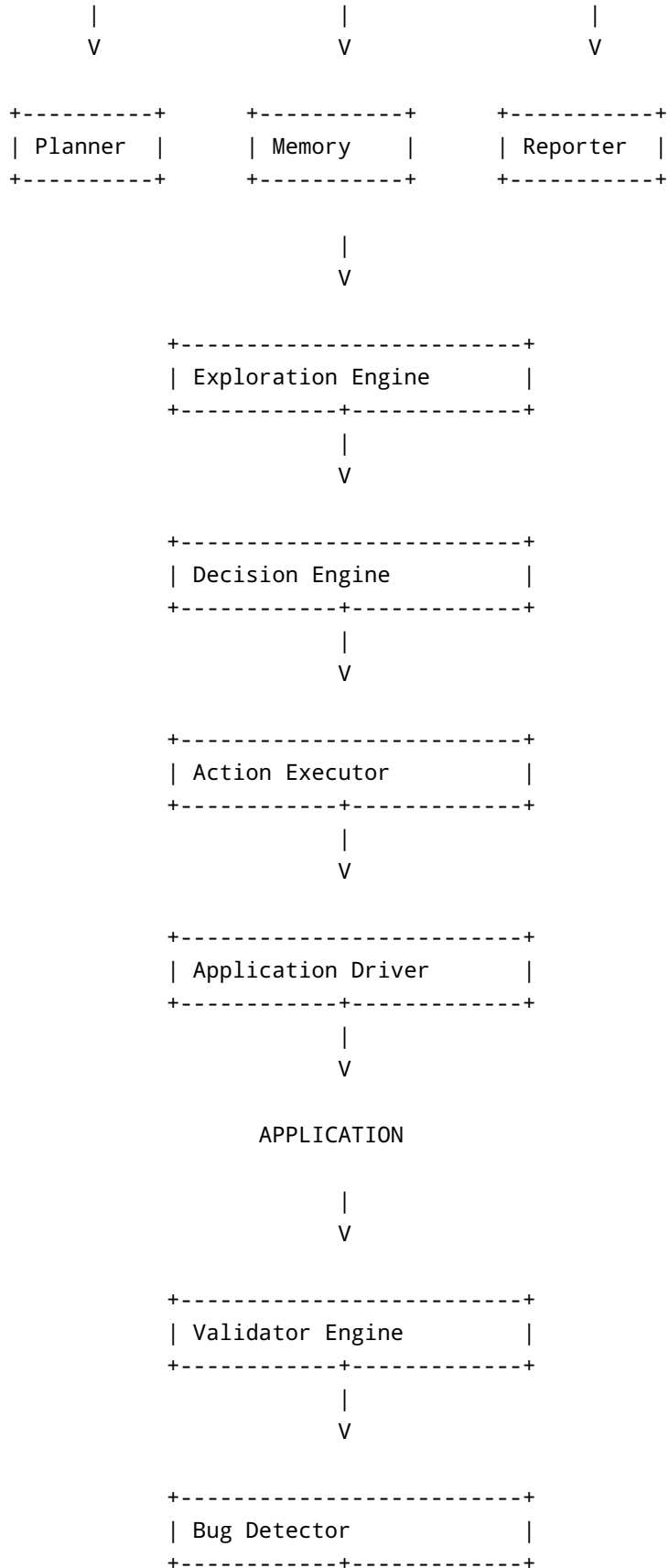
2.5 State Oriented

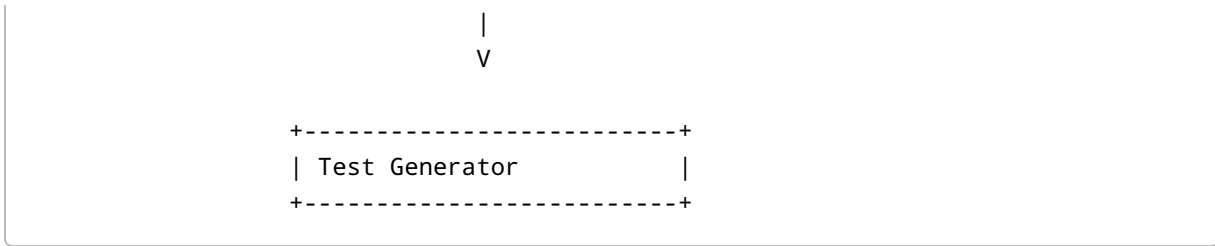
Applications are modeled as graphs consisting of:

```
States
Transitions
Actions
Events
```

3. High Level Architecture







4. System Components

The system consists of twelve major components.

5. Mission Controller

Responsibility

The Mission Controller orchestrates the entire testing session.

Example mission:

```
Explore calculator application
Find defects
Generate tests
Produce report
```

Responsibilities

- Start sessions
- Stop sessions
- Configure agent
- Define goals
- Define constraints
- Monitor progress

Interface

```
class MissionController:
```

```
def start()

def stop()

def pause()

def resume()

def get_progress()
```

6. Agent Execution Engine

Responsibility

The Agent Execution Engine controls the main execution loop.

Execution Cycle

```
Observe
  ↓
Analyze
  ↓
Reason
  ↓
Plan
  ↓
Execute
  ↓
Validate
  ↓
Learn
```

Example

```
while mission.active:

    state = observe()

    decision = think(state)
```

```
action = plan(decision)

execute(action)

validate()

learn()
```

7. Planner

Responsibility

The planner determines what to test next.

Planner Inputs

```
CurrentState
MissionGoal
History
Coverage
KnownBugs
```

Planner Outputs

```
ActionPlan
ExplorationGoal
ValidationPlan
```

Example

```
Current State:
Display=999

Goal:
Find arithmetic defects
```

Plan:
Try overflow conditions

8. Memory System

The memory system stores:

- Current session
- Previous sessions
- State graphs
- Bugs
- Test cases
- Exploration history

Memory Types

Working Memory

Current state
Current action
Current session

Episodic Memory

Previous sessions
Previous bugs
Previous states

Semantic Memory

Testing heuristics
Rules
Known bug patterns

9. Exploration Engine

Responsibility

Discovers application workflows.

Exploration Algorithms

BFS

Explore all nearby states.

DFS

Explore deeply.

Random Explorer

Choose random actions.

Heuristic Explorer

Prioritize risky states.

AI Explorer

Use LLM reasoning.

Interface

```
class Explorer:

    def next_action()

    def explore()

    def terminate()
```

10. State Graph Manager

The application is represented as a graph.

Graph Structure

```
State
 |
Action
 |
Next State
```

Example:

```
      S0
     /
    S1  S2
    |   |
    S3  S4
```

Responsibilities

- Create states
- Create transitions
- Detect loops
- Calculate coverage

- Store history
-

Interface

```
class StateGraph:  
  
    add_state()  
  
    add_transition()  
  
    find_state()  
  
    coverage()
```

11. Decision Engine

Responsibility

The decision engine determines the next action.

Inputs

```
CurrentState  
AvailableActions  
History  
MissionGoal  
Coverage
```

Outputs

```
NextAction  
Reasoning  
Confidence
```

Example

Display:
999999

Reasoning:
Large values may overflow.

Action:
*

12. Action Executor

Responsibility

Executes actions against the application.

Supported Actions

Click
DoubleClick
Input
Keyboard
Mouse
DragDrop
Scroll
Hotkey

Interface

```
execute(action)
rollback(action)
verify(action)
```

13. Application Driver Layer

The driver layer abstracts applications.

Supported Drivers

Calculator Driver

Phase 1

Desktop Driver

FlaUI
WinAppDriver
PyWinAuto

Web Driver

Playwright
Selenium

Driver Interface

```
class ApplicationDriver:

    launch()

    close()

    observe()

    get_controls()

    execute()

    screenshot()
```

```
is_alive()
```

14. Validator Engine

Responsibility

Determines if behavior is correct.

Validators

Arithmetic Validator

```
2+2=4
```

Exception Validator

```
Unhandled exceptions
```

Crash Validator

```
Application closed
```

Freeze Validator

```
Application hangs
```

Visual Validator

```
Unexpected UI changes
```

Interface

```
validate()  
  
assert_state()  
  
assert_output()
```

15. Bug Detection Engine

Responsibility

Transforms failures into defects.

Bug Categories

Crash

```
Application terminated.
```

Freeze

```
Application stopped responding.
```

Exception

```
Unhandled exception dialog.
```

Logical Bug

```
Incorrect result.
```

Visual Bug

UI corruption.

Performance Bug

Slow response.

Bug Object

```
class Bug:
    id
    severity
    category
    description
    evidence
    reproduction_steps
```

16. Test Generator

Responsibility

Generate reproducible tests.

Output Formats

Reproduction Steps

1. Click 7
2. Click /
3. Click 0
4. Click =

Gherkin

Feature: Calculator

Scenario: Divide by zero

Given calculator launched

When user presses 7

And user presses /

And user presses 0

And user presses =

Then error should appear

Reqnroll

[Given]

[When]

[Then]

Pytest

```
def test():  
    pass
```

17. Reporting Engine

Responsibilities

Generate:

- Session reports
- Bug reports
- Coverage reports
- Exploration reports
- Test reports

Report Types

HTML

Human readable

JSON

Machine readable

Markdown

Documentation

18. LLM Subsystem

Supported Providers

Ollama

Primary provider.

Supported Models

deepseek-r1:8b
qwen3:8b
qwen2.5-coder:7b
llama3.1:8b

Responsibilities

- Reasoning
- Planning

- Hypothesis generation
 - Bug explanation
 - Test generation
-

Example Prompt

You are an expert software tester.

Current state:

Display=999999

Available actions:

+, -, *, /

Suggest next action to maximize bug discovery.

19. Storage Architecture

Database

SQLite

Stored Data

Sessions

Testing sessions

States

Application states

Actions

Actions performed

Bugs

Detected bugs

Tests

Generated tests

Reports

Generated reports

20. Event Architecture

Events are used for decoupling.

Events

SessionStarted
StateObserved
ActionSelected
ActionExecuted
ValidationStarted
ValidationFinished
BugDetected
TestGenerated
SessionCompleted

Event Flow

```
ObserveState
  ↓
CreateState
  ↓
ChooseAction
  ↓
ExecuteAction
  ↓
Validate
  ↓
DetectBug
  ↓
GenerateTest
```

21. Plugin Architecture

Every subsystem is a plugin.

Plugin Categories

```
Driver Plugins
Explorer Plugins
Validator Plugins
Bug Plugins
Reporter Plugins
LLM Plugins
```

Plugin Interface

```
class Plugin:

    initialize()

    execute()
```

```
shutdown()
```

22. Execution Flow Example

Example:

```
Launch Calculator
  ↓
Observe State
  ↓
State Hash Generated
  ↓
Store State
  ↓
Discover Actions
  ↓
Choose Action
  ↓
Execute Action
  ↓
Observe Result
  ↓
Validate
  ↓
Detect Bug
  ↓
Generate Report
  ↓
Generate Test Case
```

23. Future Architecture

Future versions will support:

Multi-Agent Testing

```
Planner Agent
Explorer Agent
```

Validator Agent
Bug Agent
Reporter Agent

Distributed Testing

Multiple machines

Learning Systems

Reinforcement learning
Experience replay
Knowledge graph

Vision Models

UI understanding
Screenshot analysis
Visual bug detection

24. Architecture Decision Records

The following decisions have been made.

Decision	Choice
Language	Python
Database	SQLite
LLM	Ollama
Initial Model	DeepSeek-R1:8B
Graph Engine	NetworkX
State Storage	SQLite

Decision	Choice
Desktop Automation	PyWinAuto
Reporting	HTML + Markdown
Architecture	Plugin Based
Execution	Event Driven

25. Conclusion

The Free Form Testing Agent architecture is designed to provide:

- Modularity
- Extensibility
- Offline execution
- Autonomous exploration
- Explainable AI reasoning
- Reproducible bug discovery
- Automatic test generation

The architecture intentionally separates application interaction, exploration, reasoning, validation, and reporting so that each subsystem can evolve independently while maintaining a stable core platform.